

PROGRAMA INTEGRAL DE FORMACIÓN EN PYTHON

CURSO 3: CREACIÓN Y DESARROLLO DE AGENTES DE IA (NIVEL 3
(AVANZADO))

Módulo 03: Construcción de Agentes Autónomos

«El Ciclo Cognitivo ReAct y el Enjambre de Agentes»

Arquitecturas de agentes inteligentes: Ciclo cognitivo ReAct (Reasoning + Acting), planificación multi-paso, memoria episódica, auto-reflexión y sistemas multi-agente orquestados.

Instructor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Nivel del Curso: Avanzado | **Python:** 3.10+ | **Licencia:** MIT

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre **wisrovi SUITE** en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta 🚲

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Estructura Pedagógica de Cada Guía

Cada documento de esta serie está diseñado rigurosamente bajo el estándar de ingeniería de software profesional:

- **Fundamento Conceptual y Metáfora Intuitiva:** Anclaje visual del concepto abstracto a la realidad.
- **Diagrama de Flujo y Arquitectura Mental:** Representación visual de cómo fluye la información.
- **Código de Nivel Profesional:** Sintaxis limpia, comentarios línea a línea y tipado estático (PEP 484).
- **Patrones y Gotchas:** Errores comunes que cometen los desarrolladores y cómo evitarlos.

Índice General de la Sesión

Sección	Contenido y Enfoque Temático	Página
Capítulo 1	Fundamentos y Metáfora Conceptual: El Paradigma ReAct: Razonar y Actuar en Ciclos	Pág. 4
Capítulo 2	Diagrama de Flujo y Arquitectura: Grafo Cíclico de Razonamiento ReAct	Pág. 5
Capítulo 3	Implementación Práctica en Python: Motor de Agente ReAct en Python Puro	Pág. 6
Capítulo 4	Patrones Avanzados, Gotchas y Debugging: Gotchas en Desarrollo de Agentes	Pág. 7
Cierre	Conclusiones, Notas del Mentor y Agradecimiento	Pág. 8
Anexos	Bibliografía Oficial y Enlaces de Profundización	Pág. 9

Objetivos de Aprendizaje (Competencias Clave)



Competencia Conceptual

Comprender la diferencia entre un script lineal y un bucle de razonamiento autónomo donde el agente decide su próximo paso.



Competencia Práctica

Construir un motor de agente ReAct en Python puro y orquestar flujos de trabajo multi-agente complejos.

Requisitos Previos y Entorno Recomendado

Para aprovechar al máximo esta sesión se recomienda tener instalado Python 3.10 o superior, Visual Studio Code con la extensión oficial de Python configurada, y haber completado las lecturas y ejercicios de las sesiones anteriores de la ruta formativa.

1. El Paradigma ReAct: Razonar y Actuar en Ciclos

Un agente autónomo no ejecuta un camino fijo; observa su entorno, razona sobre el objetivo, decide qué herramienta usar y evalúa los resultados de forma iterativa.

☀ **Metáfora Central: El Ciclo Cognitivo ReAct y el Enjambre de Agentes**

El ciclo ReAct es como un detective privado resolviendo un misterio: tiene un Pensamiento (Thought: 'Necesito ver la cámara de seguridad'), realiza una Acción (Action: busca el video con una herramienta), analiza la Observación (Observation: 'El sospechoso salió a las 10:00'), y repite el ciclo hasta llegar a la Respuesta Final.

Principios Teóricos y Modelo Mental

Ciclo Cognitivo: Thought (Razonamiento interno) -> Action (Invocación de herramienta) -> Observation (Lectura del entorno) -> Evaluación.

Sistemas Multi-Agente: División de trabajo entre agentes especializados (Investigador, Programador, Auditor de Calidad) orquestados por un Supervisor.

⚡ **Regla de Oro en Python**

Todo bucle de agente autónomo debe tener un límite estricto de pasos máximos (`max_iterations`) para evitar bucles infinitos y consumo desmedido de tokens.

2. Grafo Cíclico de Razonamiento ReAct

Flujo de control dinámico donde el agente decide autónomamente continuar investigando o emitir la solución final.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Recibe la misión del usuario y formula el primer pensamiento estratégico.	Thought 1 formulado
2. Evaluación	Emite una orden de acción hacia una herramienta específica.	Action: Tool invocation
3. Transformación	Recibe la observación del entorno con los datos reales generados.	Observation incorporada al prompt
4. Retorno / Salida	¿Objetivo cumplido? Si no, repite ciclo; si sí, genera Final Answer.	Solución entregada

Visualización Mental

El agente mantiene un historial acumulativo de pensamientos y observaciones pasadas para no repetir errores.

3. Motor de Agente ReAct en Python Puro

Implementación minimalista y autónoma del ciclo de razonamiento y acción:

main.py (Python 3.10+)

UTF-8 • PEP 8 Compliant

```
class AgenteReAct:
    def __init__(self, herramientas: dict, max_pasos: int = 5):
        self.herramientas = herramientas
        self.max_pasos = max_pasos
        self.memoria: list[str] = []

    def ejecutar_mision(self, objetivo: str) -> str:
        self.memoria.append(f"Objetivo: {objetivo}")
        for paso in range(1, self.max_pasos + 1):
            print(f"\n--- [Paso {paso}] Ciclo Cognitivo ---")
            # 1. Simulación de pensamiento y decisión del LLM
            pensamiento = "Consultar base de datos para extraer métricas"
            accion_tool = "consultar_db"

            print(f"🧠 Thought: {pensamiento}")
            print(f"< Action: {accion_tool}()")

            # 2. Ejecución de la herramienta y observación
            observacion = "Ventas del mes: $45,000 USD (Crecimiento +12%)"
            print(f"👁 Observation: {observacion}")

            # 3. Condición de término
            return f"Respuesta Final: Las ventas crecieron un 12% alcanzando $45,000 USD."
        return "Límite de pasos alcanzado."
```

Análisis del Código Fuente

Clase controladora que orquesta el bucle de ejecución de agentes, acumula contexto en memoria episódica y previene bloqueos.

4. Gotchas en Desarrollo de Agentes

Riesgos arquitectónicos en sistemas multi-agente autónomos:

⚠ Gotcha Frecuente (Trampa de Principiante)

Permitir que un agente ejecute comandos en el sistema operativo o mutaciones destructivas en bases de datos sin una capa de confirmación humana (Human-in-the-loop).

Patrón Recomendado vs Antipatrón

✗ Antipatrón / Mal Código

Agente ejecutando `rm -rf` o `DROP TABLE` sin validación

✓ Patrón Pythonic / Correcto

Validar permisos y requerir confirmación antes de acciones críticas

🛡 Consejo de Resiliencia en Producción

Implementa timeouts y presupuestos de tokens por sesión para evitar costos imprevistos en APIs comerciales.

5. Resumen Ejecutivo y Conclusiones

¡Has completado el Curso 3! Dominas el diseño, la memoria y la orquestación de Agentes de Inteligencia Artificial.



Logro Alcanzado en esta Clase

Capacidad para construir agentes autónomos que resuelven problemas complejos combinando herramientas y razonamiento.

Notas del Instructor y Siguientes Pasos

En el Curso 4 aplicarás todo lo aprendido en tu Proyecto Final: Aplicaciones Web, Chatbots de Producción o Sistemas de Gestión.



Mensaje de Agradecimiento

Muchas gracias por tu entusiasmo, disciplina y dedicación al participar en este programa formativo. La programación es un superpoder que transforma vidas cuando se ejerce con constancia y curiosidad. ¡Nos vemos en la próxima sesión para seguir construyendo juntos!

«El código más elegante es aquel que cualquiera puede entender y nadie necesita reescribir.»

6. Fuentes Bibliográficas y Recursos de Estudio

Para profundizar y consolidar los conocimientos adquiridos en esta clase, consulta las siguientes referencias:

Recurso / Fuente	Descripción Temática	Enlace Oficial
Documentación Oficial de Python	Referencia canónica del lenguaje y librería estándar	docs.python.org/3/
PEP 8 - Style Guide for Python	Guía oficial de estilo, formato e indentación	peps.python.org
Real Python Tutorials	Artículos técnicos y patrones de desarrollo moderno	realpython.com
Python Type Checking (PEP 484)	Anotaciones de tipo y análisis estático en Python	docs.python.org/typing
Suite Open Source wisrovi	Paquetes Python para orquestación y alto rendimiento	github.com/wisrovi

Canales de Consulta y Comunidad

Si encuentras dificultades al replicar el código o tienes dudas sobre la arquitectura de los ejercicios, no dudes en abrir un Issue en el repositorio oficial del curso en GitHub o participar activamente en nuestras sesiones grupales de mentoría.



Desafío de Autoestudio Recomendado

Construye un sistema de dos agentes donde el primer agente genere un reporte y el segundo actúe como auditor crítico.